

WEB103 | Advanced Web Development

Advanced Web Development Fall 2026 (@ Section 1 | Wednesdays 3PM - 5PM PT)

Personal Member ID#: **154815**

Need help? Post on our [Class Slack Channel](#) or email us at support@codepath.org

Getting Started

Learning with AI ✨

▼ Course Info

Syllabus

Course Overview

Groupings

Submitting Coursework

Grading Rubrics

IDE Setup

Course Progress

▼ Individual

Unit 1

Unit 2

Unit 3

Unit 4

► Capstone Project

Schedule

Unit 2 Lab: UnEarthed, Part 2

Overview

As Lady Gaga *probably* once said, "A gift is worth a thousand words." So it's time to take UnEarthed to the next level, and step away from using hardcoded data! In this unit's lab, you'll create a Postgres database that hooks up to the UnEarthed web app. You'll modify the fronted code to accept information from the `gifts` table in the database. The days of serving static HTML are behind you!

Goals

By the end of this lab you will be able to...

- ☐ Setup a cloud PostgreSQL database on Render
- ☐ Connect to a PostgreSQL database on Render from a web application and from the command line
- ☐ Create a data table called `gifts` with columns for `id`, `name`, `pricepoint`, `audience`, `image`, `description`, `submittedby`, and `submittedon`
- ☐ Write SQL queries to create tables, insert data, and read data from a database

Features

Required Features

- ☐ A Postgres database is created to store the data for the web app.
- ☐ The server creates at least one new table in the Postgres database.
- ☐ The server populates the table with rows of data.
- ☐ The server is connected to a Postgres database.
- ☐ The server queries the table to insert and retrieve data.



Stretch Features

- ☐ Style web application using [Picocss](#)

Resources

- [Express API Reference](#)
- [Render Docs](#)

Lab Instructions

Required Features

If you weren't able to complete the [Week 1 Lab](#), download the [Week 1 Lab Exemplar](#) for a fresh start on the Week 2 Lab!

Step 0: Create and connect to Postgres database using Render

Install dependencies

To use a Postgres database in our Express server, we're going to need a couple things - the database itself and a way to connect to it. We'll be using [Render](#) to create and host our Postgres database and

the `pg` package to connect to it. We'll also verify our database contents from the command line with `psql`. So let's get started!

✨ AI Opportunity

► Use AI to explain code concepts → Express `pg`

- ☐ Navigate to your `server` directory and run the command `npm install pg dotenv`.
- ☐ In your `server` directory, create a new directory called `config`. This directory will store files related to server and database configurations.
- ☐ Inside the `config` directory, create a JavaScript file called `database.js`.
- ☐ In `database.js`, import `pg` from the `pg` package.

```
import pg from 'pg'
```

Connect to a Postgres database

One common way to connect to a Postgres database is using a connection pool. A **connection pool** is a cache of database connections maintained so that the connections can be reused when needed, rather than being opened and closed for each database transaction.

To do this, we need to create a configuration object and pass it to the `pg.Pool` constructor. Then we can use the connection pool object to query the database by acquiring a client from the pool, executing queries on the client, and then releasing the client back to the pool.

- ☐ Create a new variable called `config` and set it to an empty object. This will be our configuration object for the Postgres database connection.

```
const config = {  
  
}
```

- ☐ Set the `config` object's properties for the `user`, `password`, `host`, `port`, and `database` to their respective environment variables (we'll set these up soon). Also include `ssl` configuration to allow for self-signed certificates.

```
const config = {  
  user: process.env.PGUSER,  
  password: process.env.PGPASSWORD,  
  host: process.env.PGHOST,  
  port: process.env.PGPORT,  
  database: process.env.PGDATABASE,  
  ssl: {  
    rejectUnauthorized: false  
  }  
}
```

- ☐ Create a new variable called `pool` that is set to a new `pg.Pool` object and pass the `config` object as the argument to its constructor. Export this variable.

```
export const pool = new pg.Pool(config)
```

✨ AI Opportunity

- Use AI to test your understanding of a topic → alternate solutions

Environment variables

We told our `config` object to use environment variables for its configuration, so we need to provide those environment variables. Let's do that next.

- ☐ In the `server` directory, create a file called `.env`
- ☐ Enter the following variables in the `.env` file (we'll fill in the values soon):

```
PGDATABASE=  
PGHOST=  
PGPASSWORD=  
PGPORT=  
PGUSER=
```

- ☐ Add `.env` to the `.gitignore` file in your project's root directory.

✨ AI Opportunity

- Use AI to explain code concepts → `.gitignore`

We need to tell the server how to find and read this `.env` file so it can *actually* use these values in our database configuration. That's where the `dotenv` package comes in.

The `dotenv` package is used to load environment variables from a `.env` file into `process.env`. This allows developers to store sensitive information, such as database credentials, API keys, and other configuration options, in a separate file that is not committed to version control.

- ☐ In your `config` directory, create a new file called `dotenv.js`.
- ☐ In the `dotenv.js` file, import `dotenv` from the `dotenv` package and use `config()` to specify the path for the `.env` file.

```
import dotenv from 'dotenv'
dotenv.config({ path: '../.env' })
```

✨ AI Opportunity

► Use AI to explain code concepts → `.dotenv`

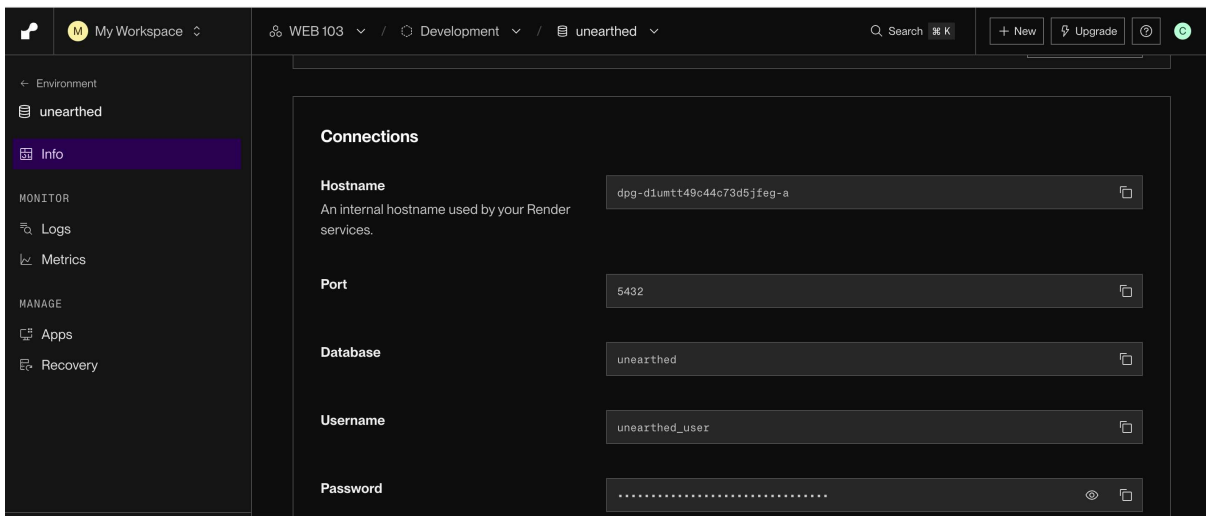
- ☐ In `server.js`, import the `dotenv.js` file right after the `express` import.

Create PostgreSQL database on Render

- ☐ Register for a [Render account](#).
- ☐ From [your dashboard](#), select **Create a Postgres database**.
- ☐ On the New Postgres page, give your instance the name **unearthed**, and select **+ Create a project**
- ☐ Create your project with the name **WEB 103** and the environment name **Development**.
- ☐ Back on the New Postgres page, scroll down to Plan Options and make sure you have the **Free** option selected before clicking **Create Database**.
- ☐ After the PostgreSQL database instance has loaded, scroll down to the **Connections** subheading. In your `.env` file, populate the variables with their corresponding values in Render as shown below:

```
PGDATABASE="{Database}"
PGHOST="{Hostname}.oregon-postgres.render.com"
PGPASSWORD="{Password}"
PGPORT={Port}
PGUSER="{Username}"
```

Here's where you can find those variables in Render:



Checkpoint 0: Your `database.js` file should look like the one above and your `.env` file should have variables that look like this (these are fake, but yours should look similar):

```
PGDATABASE="unearthed"
PGHOST="dpg-d1umtt49c44c73d5jfg-a.oregon-postgres.render.com"
PGPASSWORD="rPaThAmDRXMDdLI75B0SxyahIr"
PGPORT=5432
PGUSER="unearthed_user"
```

Step 1: Create and seed a database table

Now that we have a Render database and we can connect to it in our server, let's create a `gifts` table and send the data in our `gifts.js` to the database.

The `gifts` table has the following structure:

attribute	type	primary key
id	integer	✓
name	text	
pricepoint	text	
audience	text	
image	text	
description	text	
submitted by	text	
submitted on	date	

- ☐ In the `server\config` directory, create a new file called `reset.js`. This file will create a `gifts` table and load the data we already have into the database.

☐ Import the following:

- ☐ the `pool` from `database.js`
- ☐ the `dotenv.js` file
- ☐ the `giftData` from `data/gifts.js`

Create the `gifts` table

Now that we have the file set up, let's start with creating the `gifts` table.

- ☐ In `reset.js`, create an asynchronous function called `createGiftsTable()`.
- ☐ Create a new variable called `createTableQuery` that is set to a SQL query to drop the `gifts` table if it exists and create a new table called `gifts` if it doesn't exist.

```
const createTableQuery = `
  DROP TABLE IF EXISTS gifts;

  CREATE TABLE IF NOT EXISTS gifts (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    pricePoint VARCHAR(10) NOT NULL,
    audience VARCHAR(255) NOT NULL,
    image VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    submittedBy VARCHAR(255) NOT NULL,
    submittedOn TIMESTAMP NOT NULL
  )
`
```

✨ AI Opportunity

► Use AI to understand provided code

- ☐ Create a `try/catch` block.
- ☐ In the `try` block, query the `pool` with the `createTableQuery`.

```
try {
  const res = await pool.query(createTableQuery)
  console.log('🎉 gifts table created successfully')
}
```

- ☐ In the `catch` block, log the error.

```
catch (err) {
  console.error('⚠️ error creating gifts table', err)
}
```

Load the data into the table

We already have the data we need the table to start with in our `data/gifts.js` file. Let's send this data over to the database.

- ☐ In `reset.js`, create another asynchronous function called `seedGiftsTable()`.
- ☐ Call the `createGiftsTable()` function and wait for the response.

```
const seedGiftsTable = async () => {  
  await createGiftsTable()  
}
```

- ☐ Traverse the `giftData`.

```
giftData.forEach((gift) => {  
  
})
```

- ☐ Inside the `forEach`, create a new variable called `insertQuery` that is set to a SQL query to insert the values into the `gifts` table.

```
giftData.forEach((gift) => {  
  const insertQuery = {  
    text: 'INSERT INTO gifts (name, pricePoint, audience, image, description, submittedBy, submittedOn)  
  }  
})
```

- ☐ Create another variable called `values` that is set to an array of the gift attributes.

```
const values = [  
  gift.name,  
  gift.pricePoint,  
  gift.audience,  
  gift.image,  
  gift.description,  
  gift.submittedBy,  
  gift.submittedOn  
]
```

- ☐ Query the `pool` with the `insertQuery` and `values`. If there is an error, log the error.

```
pool.query(insertQuery, values, (err, res) => {
  if (err) {
    console.error('⚠ error inserting gift', err)
    return
  }

  console.log(`✅ ${gift.name} added successfully`)
})
```

- ☐ At the end of the file, call the `seedGiftsTable()` function.

Update the `package.json` file

We need to make sure the database is reset when the server is started, which means we need to update the `"start"` script in the server's `package.json`.

- ☐ In the `server/package.json` file, add the following to the `"scripts"`:

```
"reset": "cd config && node --require dotenv/config reset.js"
```

- ☐ Update the `"start"` script to the following:

```
"start": "npm run reset && nodemon --require dotenv/config server.js",
```

- ☐ Test your scripts and database by running `npm run reset` and `npm run start`. You should see your table and rows being created successfully:

```
server git: (kp/test-lab2) ✖ npm run reset
> server@1.0.0 reset
> cd config && node --require dotenv/config reset.js

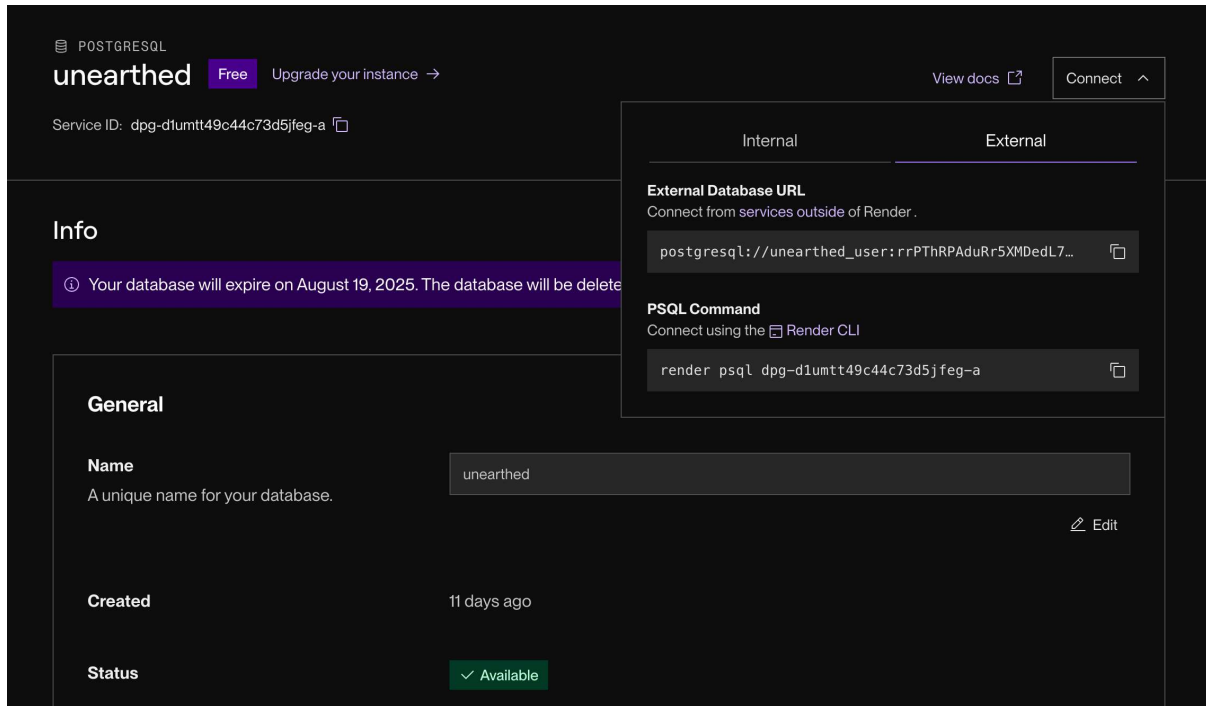
[dotenv@17.2.1] injecting env (5) from ../.env -- tip: ⚙️ suppress all logs
with { quiet: true }
🎉 gifts table created successfully
✅ Disco Ball Candle added successfully
✅ Nike Panda Dunks added successfully
✅ Nellsie Sunset Lamp Projection added successfully
✅ Razer Kitty Headset added successfully
✅ Crosley Discover Turntable added successfully
✅ Nintendo Switch Lite added successfully
✅ Lego Flower Bouquet Kit added successfully
✅ Brandon Blackwood Parker Bag added successfully
✅ Frigidaire Retro Mini Fridge added successfully
```

Utilize `psql` to verify table structure and data insertion

Next, we will connect to our database from the command line using the `psql` tool. By sending specific `psql` commands and SQL statements, we will be able to confirm that our database seeding

procedure was successful. This method of confirmation compliments the prior script output by providing low-level insight into our data - if this `psql` output is correct, we know **for sure** that everything was created as expected.

- ☐ If you haven't already, download Postgres to your development machine using these [links](#). This will include the `psql` utility.
- ☐ Open your [Render dashboard](#) and select your Postgres instance. Find the **Connect** dropdown button in the upper right, click it, and select the **External** tab.



- ☐ Copy the **External Database URL** connection string. This string is an address you can use to connect to your specific Postgres instance. It contains sensitive information: your username and password for Postgres! Don't share this information.
- ☐ Open a terminal window and type `psql` followed by a space, and then paste in your connection string and hit enter. You should see some output, concluding with a prompt for interfacing with your database (`unearthed=>`). You're in!
- ☐ Now, we will retrieve a list of **tables** present in your database. Type `\d` and hit enter. The output should look similar to the following:

```
          List of relations
Schema |   Name   | Type   | Owner
-----+-----+-----+-----
public | gifts    | table  | unearthed_user
public | gifts_id_seq | sequence | unearthed_user
(2 rows)
```

- ☐ If you see that the `gifts` table has been created successfully, we are good to proceed with further analysis. Otherwise, revisit the database seeding steps above and make sure you see the expected output from those steps.

- ☐ Next, we will verify the **columns** that were created for the `gifts` table. Type `\d gifts` and hit enter. This output lists the names, types, and other configurations for all of the columns in the `gifts` table. Look for something like this:

Table "public.gifts"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('gifts_id_seq'::regclass)
name	character varying(255)		not null	
pricepoint	character varying(10)		not null	
audience	character varying(255)		not null	
image	character varying(255)		not null	
description	text		not null	
submittedby	character varying(255)		not null	
submittedon	timestamp without time zone		not null	

Indexes:

"gifts_pkey" PRIMARY KEY, btree (id)

- ☐ Finally, we will verify the **rows** that have been inserted into `gifts`. This command is a SQL query, as opposed to a `psql` command. That means we need to make sure that we include a semicolon (`;`) at the end of our query. (If this were a write operation, we would be implementing [extra steps](#) to be able to carefully proceed or roll back the operation. Since this is a read operation, we'll simply proceed with the query.) Type `SELECT * FROM gifts;` and hit enter. Here's a snippet of what you can expect to see:

id	name	pricepoint	audience	image
1	Disco Ball Candle	\$	Candle Lovers	https://tinyurl.com/39xc9h6
2	Nintendo Switch Lite	\$\$	Gamers	https://tinyurl.com/2cdfe59
3	Frigidaire Retro Mini Fridge	\$	Foodies	https://tinyurl.com/dzyhec8
4	Razer Kitty Headset	\$\$	Gamers	https://tinyurl.com/4pt84wr
5	Nike Panda Dunks	\$\$	Sneakerheads	https://tinyurl.com/y9bzbv8x
6	Lego Flower Bouquet Kit	\$	Green Thumbs	https://tinyurl.com/ydpv2bt
7	Nellsie Sunset Lamp Projection	\$	Sungazers	https://tinyurl.com/y76zsc3
8	Brandon Blackwood Parker Bag	\$\$	Fashionistas	https://tinyurl.com/2n6pszw
9	Crosley Discover Turntable	\$	Music Lovers	https://tinyurl.com/4vycxm

(9 rows)

- ☐ Finally, to exit your Postgres connection, type `\q` then hit Enter or hit `Ctrl-d` while at your `psql` prompt (`unearthed =>`). This will drop you back to your terminal prompt.

🔴 **Checkpoint 1:** The `psql` output above gives you extra confidence that your tables and data are set up just the way you expect! Feel free to revisit these operations at any time in the future to confirm that data has been inserted correctly.

► Your `reset.js` file should look like this 🙌

Step 2: Create routes and controllers

Let's start by building routes and controllers, which return resources from the `gifts` table in the database.

- ☐ In the `server` directory, create a directory called `controllers`.
- ☐ In the `controllers` directory, create a JavaScript file called `gifts.js` to store the controller functions to perform CRUD operations associated with the `gifts` table.
- ☐ In `gifts.js`, import the `pool` from `../config/database.js`.

```
import { pool } from '../config/database.js'
```

Let's create the `getGifts()` function.

- ☐ Define an asynchronous function called `getGifts()` that has two parameters `req` and `res`.

```
const getGifts = async (req, res) => {  
  }  
}
```

- ☐ Inside the function body, add a `try/catch` block to help with error handling.
- ☐ In the `try` block, make a query to the database to request all rows of data from the `gifts` table.

✨ AI Opportunity

► Use AI to brainstorm ideas for your code → writing a SQL query

► Want to double-check your code? Compare what you have to this 

- ☐ Next, send status `200` and the `rows` from the results from the query to indicate that the request was successful.

```
res.status(200).json(results.rows)
```

- ☐ In the `catch` block, send status `409` and return an error message.

```
catch (error) {  
  res.status(409).json( { error: error.message } )  
}
```

✨ AI Opportunity

► Use AI to explain unfamiliar terms → the `409` status code

☐ Lastly, export the `getGifts()` function.

► Your `gifts.js` should look like this 📌

☐ In `routes/gift.js`, import the `gifts.js` controller as `GiftsController`.

```
import GiftsController from '../controllers/gifts.js'
```

☐ Update the `/` route to use the `getGifts` function.

```
router.get('/', GiftsController.getGifts)
```

💡 Note: This means you can also delete the line `import giftData from '../data/gifts.js'` since we are serving the data from the database instead of directly from this file.

📌 **Checkpoint 2:** Your app should be serving gifts just as it was before! Isn't using a database so much easier? Your database makes your app so much more scalable. Now `Unearthed` has the potential to launch!

🎉🎉🎉 Congratulations, you've completed this lab! 🎉🎉🎉